

Hanam_MC Unity 프로젝트 기술 보고서 v2

최초 작성일: 2025-11-28 최종 수정일: 2026-03-11 프로젝트명: Hanam_MC (하남시정신건강복지센터 AI생각돌봄 소프트웨어) 분석 대상: Assets/01. Script 폴더 내 210개 C# 스크립트 기술 스택: Unity 2022+, LiteDB, [BCrypt.Net](#), TextMeshPro, STT(음성인식)

목차

- [프로젝트 개요](#)
- [시스템 아키텍처](#)
- [폴더 구조 분석](#)
- [DB 레이어 상세 분석](#)
- [핵심 모듈별 상세 분석](#)
- [데이터 모델 및 스키마](#)
- [씬 및 UI 플로우](#)
- [부가 시스템](#)
- [적용된 디자인 패턴](#)
- [보안 및 검증](#)
- [서버 연동 가이드](#)
- [결론 및 개선 제안](#)

1. 프로젝트 개요

1.1 시스템 목적

Hanam_MC는 **하남시정신건강복지센터의 AI생각돌봄 소프트웨어**로, Unity 기반의 교육용 게임 애플리케이션입니다. 사용자가 다양한 테마(Director, Gardener 등)의 문제를 풀면서 학습하고, 보상 아이템을 수집하며, 음성인식(STT) 기반 인터랙션을 통해 정신건강 돌봄 활동에 참여하는 시스템입니다.

1.2 주요 기능

기능	설명
회원 관리	회원가입, 로그인, 역할 기반 접근 제어 (USER/ADMIN/SUPERADMIN)
문제 풀이	테마별 10개 문제, 단계별(Step) 진행, 다양한 인터랙션
진행도 추적	사용자별 풀이 기록, Attempt 로그, 완료 상태 관리
보상 시스템	문제 완료 시 인벤토리 아이템 지급
관리자 기능	사용자 검색, 진행 상황 모니터링, 피드백 제공
음성인식(STT)	음성 입력 기반 키워드 매칭 및 인터랙션
이펙트 시스템	문제별 시각 효과, 애니메이션, 파티클 연출
가상 키보드	터치 기반 한글/영문/숫자 입력 지원
다국어 지원	CSV 기반 한국어/영어 로컬라이제이션

1.3 기술 스택

분류	기술
게임 엔진	Unity 2022+
프로그래밍 언어	C#
데이터베이스	LiteDB (임베디드 NoSQL)
비밀번호 암호화	BCrypt.Net (WorkFactor=10)
UI 텍스트	TextMeshPro
음성인식	STT (Speech-to-Text) 모듈
데이터 설정	CSV 기반 DataTable, ScriptableObject

2. 시스템 아키텍처

2.1 전체 레이어 구조

Presentation Layer (UI Components, Controllers, Scene Managers) LoginFormUI, HomeSceneUI, ProblemStepBase, Effect 시스템
Application Layer (Business Logic Services) AuthService, ProgressService, RewardService, AdminService
Data Access Layer (Repositories) UserRepo, ProgressRepo, InventoryRepo, ProblemRepo 등
Infrastructure Layer (Database Gateway, Initializer) DBGateway, DBHelper, DatabaseInitializer
Support Systems STT, SoundManager, VirtualKeyboard, LocalizedTable
Database Layer LiteDB (mc.db)

2.2 핵심 설계 원칙

- 1. 계층 분리 (Layered Architecture): UI → Controller → Service → Repository → DB의 명확한 책임 분리
- 2. 의존성 역전 원칙 (DIP): 상위 레이어가 하위 레이어의 인터페이스에만 의존
- 3. 단일 책임 원칙 (SRP): 각 클래스가 하나의 책임만 담당
- 4. 인터페이스 분리 원칙 (ISP): 역할별로 세분화된 인터페이스 정의
- 5. 템플릿 메서드 패턴: ProblemStepBase를 통한 공통 흐름 정의

3. 폴더 구조 분석

3.1 전체 폴더 구조

```
Assets/01. Script/
├─ Bootstrap.cs                # 앱 진입점, DB 인덱스 초기화, 로컬라이제이션 로드
├─ Core/                      # 핵심 매니저 (3개 파일)
│  ├─ SessionManager.cs       # 세션 상태 관리 (PlayerPrefs 기반)
│  ├─ SceneNavigator.cs       # 씬 전환 관리 (히스토리, 페이드 효과)
│  └─ GameManager.cs          # 앱 수준 유틸리티 (종료, 로그아웃)
├─ Data/                      # 데이터 레이어 (20+ 파일)
│  ├─ Model.cs                # 데이터 모델 정의
│  ├─ DataService.cs          # 서비스 싱글톤 허브 (DI 조립)
│  ├─ DBHelper.cs             # DB 연결 헬퍼
│  ├─ DatabaseInitializer.cs   # DB 인덱스 초기화
│  ├─ DBGateway/              # DB 접근 게이트웨이
│  │  ├─ DBGateway.cs
│  │  └─ DBGateway.Core.cs
│  ├─ Repository/             # 데이터 접근 레이어 (6개)
│  │  ├─ UserRepository.cs
│  │  ├─ ProgressRepository.cs
│  │  ├─ InventoryRepository.cs
│  │  ├─ ProblemRepository.cs
│  │  ├─ ResultRepository.cs
│  │  └─ FeedbackRepository.cs
│  ├─ LocalProgressService.cs  # 진행도 서비스
│  ├─ LocalRewardService.cs    # 보상 서비스
│  ├─ LocalAdminDataService.cs # 관리자 서비스
│  ├─ LocalProblemQueryService.cs # 문제 조회 서비스
│  ├─ LocalResultQueryService.cs # 결과 조회 서비스
│  └─ UserSearchUtility.cs     # 한글 초성 검색 유틸리티
├─ Service/                   # 인증 서비스 (5개 파일)
│  ├─ AuthService.cs           # 로그인/회원가입
│  ├─ AuthValidator.cs         # 입력 검증
│  ├─ AuthConfig.cs            # ScriptableObject 인증 설정
│  ├─ AuthUIText.cs            # UI 텍스트 ScriptableObject
│  └─ AdminService.cs          # 관리자 기능
├─ RegisterScene/             # 로그인/회원가입 씬 (6개 파일)
│  ├─ LoginFormUI.cs           # 로그인 UI
│  ├─ LoginController.cs       # 로그인 로직
│  ├─ SignupFormUI.cs          # 회원가입 UI
│  ├─ SignupController.cs      # 회원가입 로직
│  ├─ RegisterTabsController.cs # 탭 전환
│  └─ Result.cs                # Result 모노이드
├─ HomeScene/                 # 메인 화면 (4개 파일)
│  ├─ HomeSceneUI.cs           # 메인 UI (환영 메시지, 로그아웃)
│  ├─ ThemePanelsController.cs  # 테마/레벨 선택, 잠금 해제 로직
│  ├─ ThemePanelUI.cs          # 개별 테마 UI (10개 문제 버튼)
│  └─ ProblemSession.cs         # 문제 선택 세션 (DemoMode 지원)
├─ ProblemScene/              # 문제 풀이 (50+ 파일)
│  ├─ ProblemSceneController.cs # 테마/문제 라우팅, 옵션 패널
│  ├─ ProblemContext.cs         # ScriptableObject 문제 컨텍스트
│  ├─ StepFlowController.cs     # Step 순서 관리, BGM 제어
│  ├─ StepCompletionGate.cs     # 진행도 추적 위젯
│  ├─ StepKeyConfig.cs          # Step 식별자 구조체
│  └─ StepErrorPanel.cs         # 에러 메시지 표시
```

- └─ StepBases/ # Step 기본 클래스
 - └─ ProblemStepBase.cs # 추상 베이스
 - └─ MultipleChoiceStepBase.cs # 객관식
 - └─ RandomCardSequenceStepBase.cs # 랜덤 카드 순서
 - └─ InventoryDropTargetStepBase.cs # 드래그 앤 드롭
 - └─ CommonRewardStep.cs # 공통 보상 연출
- └─ StepComponent/ # Step UI 컴포넌트
 - └─ UIDropBoxArea.cs # 드래그 앤 드롭 영역
 - └─ UICardFlip.cs # 카드 뒤집기
 - └─ UILineConnector.cs # 애니메이션 라인
- └─ Director/ # Director 테마 문제
 - └─ Problem1/ ~ Problem10/
- ResultScene/ # 관리자 화면 (4개 파일)
 - └─ AdminUserBrowserController.cs # 사용자 검색/관리
 - └─ AdminUserBrowserUI.cs # 관리자 UI
 - └─ AdminUserItemUI.cs # 사용자 항목 UI
 - └─ AdminUserCommentPanel.cs # 피드백 패널
- Inventory/ # 인벤토리 (3개 파일)
 - └─ RewardInventoryPanel.cs # 보상 화면
 - └─ StepInventoryPanel.cs # Step 내 인벤토리
 - └─ StepInventoryItem.cs # 개별 슬롯 (드래그 지원)
- Effect/ # 이펙트 시스템 (66개 파일)
 - └─ Common/ # 공통 이펙트 (25+개)
 - └─ EffectControllerBase.cs # 이펙트 베이스 클래스
 - └─ AppearAnimation.cs # 등장 애니메이션
 - └─ BadgeGlowPulse.cs # 배지 발광
 - └─ ButtonHover.cs # 버튼 호버
 - └─ CardFlip.cs # 카드 플립
 - └─ DragScaleFeedback.cs # 드래그 피드백
 - └─ FloatingAnimation.cs # 떠다니는 효과
 - └─ GlowEffect.cs # 발광 효과
 - └─ PopupSpring.cs # 팝업 스프링
 - └─ PulseAnimation.cs # 맥동 효과
 - └─ RadialBurstEffect.cs # 방사형 폭발
 - └─ RewardParticleBurst.cs # 보상 파티클
 - └─ ShakeTrigger.cs # 흔들림 효과
 - └─ SlideUpFadeIn.cs # 슬라이드 페이드인
 - └─ SparkleEffect.cs # 반짝임 효과
 - └─ ...
 - └─ ProblemScene/ # 문제별 이펙트 (40+개)
 - └─ Director/
 - └─ Problem1/ # P1 Step별 이펙트 컨트롤러
 - └─ Problem2/ ~ Problem10/
 - └─ ...
- DataTable/ # 데이터 테이블 (3개 파일)
 - └─ LocalizedTable.cs # CSV 기반 로컬라이제이션
 - └─ RewardTable.cs # 보상 설정
 - └─ CsvUtil.cs # CSV 파싱 유틸리티
- STT/ # 음성인식 (5개 파일)
 - └─ STTManager.cs # 음성인식 관리자
 - └─ STTHelper.cs # STT 유틸리티
 - └─ STTButton.cs # STT 버튼 UI
 - └─ KeywordMatcher.cs # 키워드 매칭
 - └─ STTTester.cs # 디버그 테스터
- Sound/ # 사운드 (1개 파일)
 - └─ SoundManager.cs # BGM 재생/정지 (싱글톤)
- Keyboard/ # 가상 키보드 (6개 파일)

```

├─ VirtualKeyboard.cs          # 키보드 코어
├─ VirtualKeyboardController.cs # 입력 관리
├─ VirtualKeyboardInputField.cs # InputField 연결
├─ VirtualKeyboardKey.cs       # 개별 키
├─ VirtualNumpad.cs           # 숫자 패드
├─ VirtualKeyboardGenerator.cs # 레이아웃 생성
├─ Debug/                      # 디버그 (2개 파일)
├─ DebugShortcutController.cs  # 개발용 단축키
├─ DebugStepController.cs      # Step 디버그 네비게이션
├─ Editor/                     # 에디터 도구
├─ ...

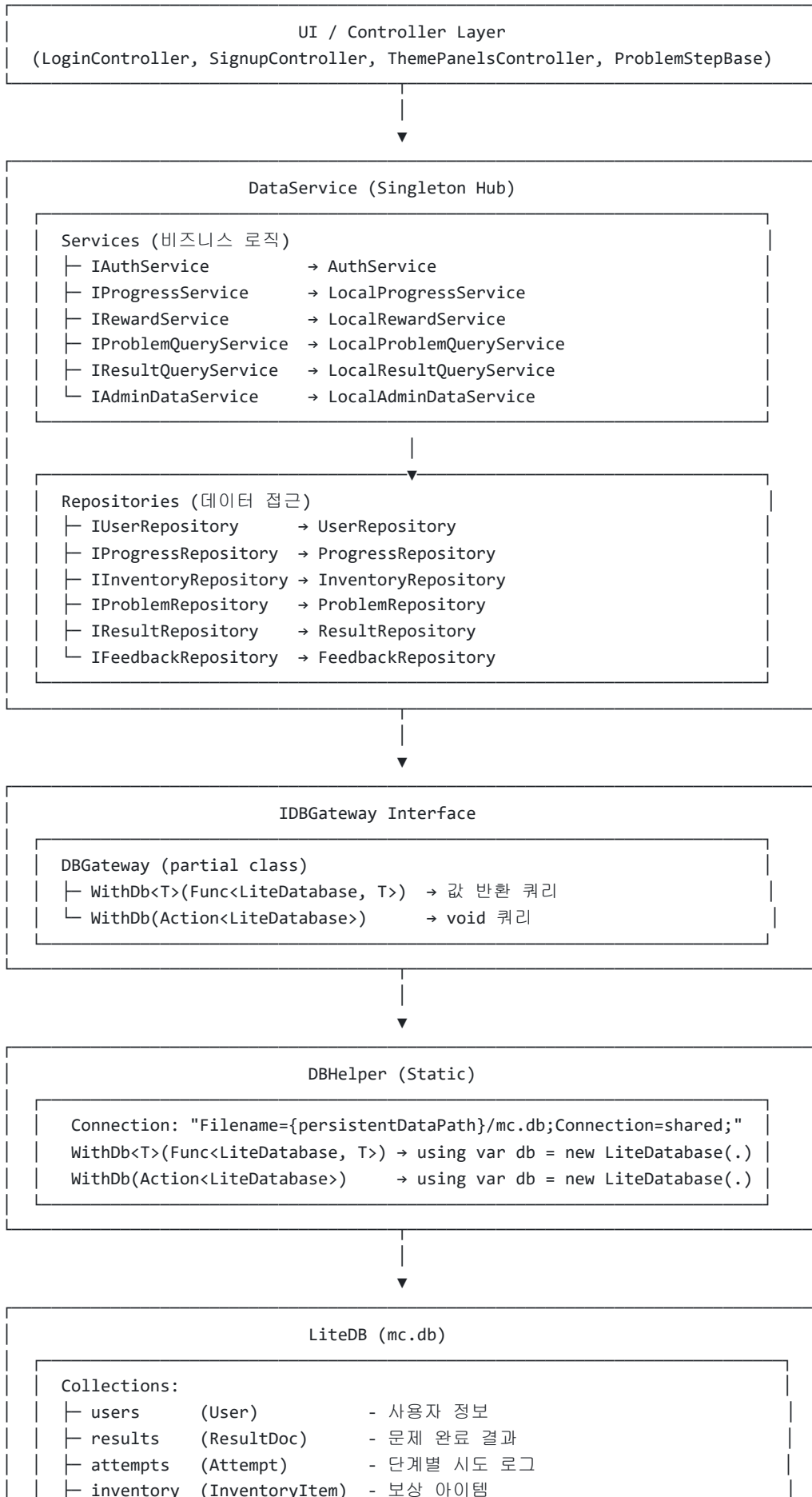
```

3.2 파일 통계

폴더	파일 수	역할
루트	1개	앱 초기화 (Bootstrap)
Core	3개	세션, 씬 전환, 앱 매니저
Data	20+개	데이터 모델, Repository, Service, DB
Service	5개	인증, 검증, 설정, 관리자
RegisterScene	6개	로그인/회원가입 UI 및 로직
HomeScene	4개	메인 화면, 테마/문제 선택
ProblemScene	50+개	문제 풀이 로직, Step 구현
ResultScene	4개	관리자 사용자 브라우저
Inventory	3개	인벤토리 UI 및 아이템
Effect	66개	공통 이펙트 + 문제별 이펙트
DataTable	3개	CSV 로컬라이제이션, 보상 설정
STT	5개	음성인식
Sound	1개	BGM 관리
Keyboard	6개	가상 키보드
Debug	2개	개발 도구
Editor	1+개	Unity 에디터 확장
총계	약 210개	

4. DB 레이어 상세 분석

4.1 전체 DB 아키텍처



└─ sessions	(SessionRecord)	- 세션 기록
└─ problems	(Problem)	- 문제 정보
└─ feedback	(Feedback)	- 관리자 피드백

4.2 DB 인덱스 초기화 (DatabaseInitializer)

앱 시작 시 Bootstrap.cs 에서 DatabaseInitializer.InitializeIndexes() 를 호출하여 모든 컬렉션의 인덱스를 일괄 생성합니다. 기존에 각 Repository 메서드에서 매번 EnsureIndex 를 호출하던 방식을 중앙화한 것입니다.

```
public static class DatabaseInitializer
{
    public static void InitializeIndexes()
    {
        DBHelper.WithDb(db =>
        {
            // User 인덱스
            var users = db.GetCollection<User>("users");
            users.EnsureIndex(x => x.Email, true);
            users.EnsureIndex(x => x.Id, true);
            users.EnsureIndex(x => x.LowerName);
            users.EnsureIndex(x => x.Role);

            // Result, Attempt, Inventory, Problem 등 인덱스 ...
        });
    }
}
```


4.3 의존성 주입 (Dependency Injection) 흐름

DataService.Awake()에서 조립

```
void Awake()
{
    // 1. 싱글톤 설정
    if (Instance != null) { Destroy(gameObject); return; }
    Instance = this;
    DontDestroyOnLoad(gameObject);

    // 2. DBGateway 생성 (최하위 레이어)
    Db = new DBGateway();
    var dbCore = (IDBGateway)Db;

    // 3. Repository 생성 (DBGateway 주입)
    InventoryRepository = new InventoryRepository(dbCore);
    UserRepository       = new UserRepository(dbCore);
    ProgressRepository   = new ProgressRepository(dbCore);
    ProblemRepository    = new ProblemRepository(dbCore);
    ResultRepository     = new ResultRepository(dbCore);
    FeedbackRepository   = new FeedbackRepository(dbCore);

    // 4. Service 생성 (Repository들 주입)
    Auth = new AuthService(UserRepository);

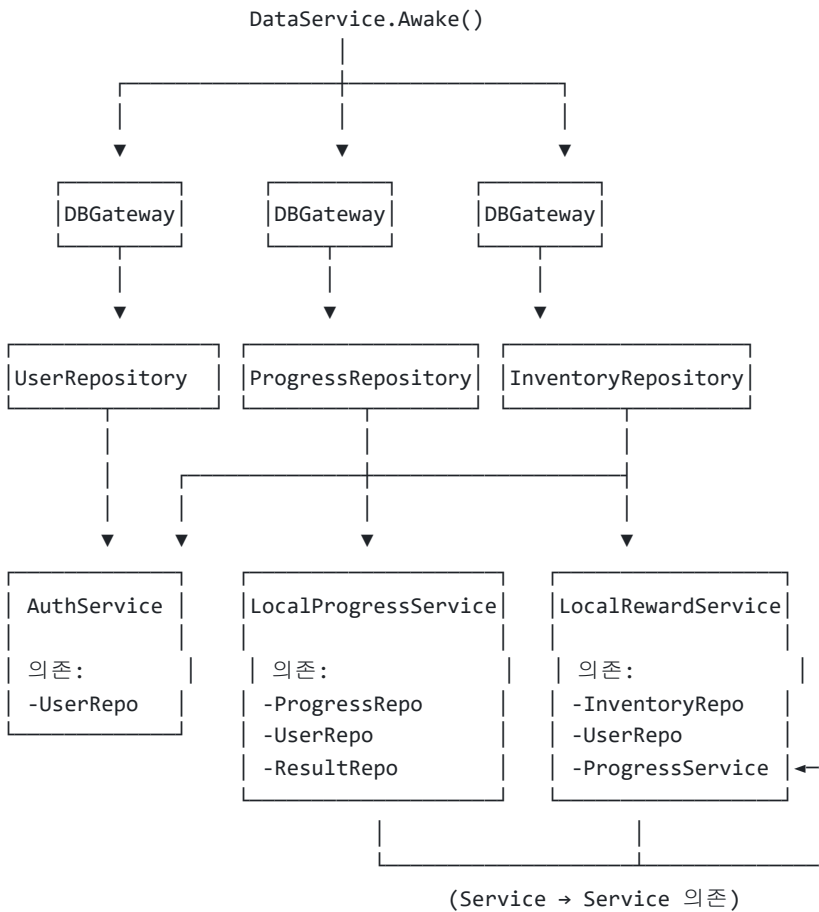
    Progress = new LocalProgressService(
        ProgressRepository, UserRepository, ResultRepository
    );

    Reward = new LocalRewardService(
        InventoryRepository, UserRepository, Progress
    );

    Problems = new LocalProblemQueryService(ProblemRepository);
    Results  = new LocalResultQueryService(ResultRepository);

    Admin = new LocalAdminDataService(
        UserRepository, ResultRepository, FeedbackRepository
    );
}
```

의존성 주입 다이어그램



4.4 각 레이어별 코드 분석

Layer 1: DBHelper (최하위 - 인프라)

```

public static class DBHelper
{
    static string DBPath => Path.Combine(Application.persistentDataPath, "mc.db");
    static string LitDB_Connection => $"Filename={DBPath};Connection=shared;";

    public static T WithDb<T>(Func<LiteDatabase, T> f)
    {
        Directory.CreateDirectory(Application.persistentDataPath);
        using var db = new LiteDatabase(LitDB_Connection);
        return f(db);
    }

    public static void WithDb(Action<LiteDatabase> f)
    {
        Directory.CreateDirectory(Application.persistentDataPath);
        using var db = new LiteDatabase(LitDB_Connection);
        f(db);
    }
}

```

역할: LiteDB 연결 생성/해제를 using 패턴으로 자동 관리, 연결 문자열 중앙 관리

Layer 2: IDBGateway / DBGateway

```
public interface IDBGateway
{
    T WithDb<T>(Func<LiteDatabase, T> func);
    void WithDb(Action<LiteDatabase> action);
}

public partial class DBGateway : IDBGateway
{
    public T WithDb<T>(Func<LiteDatabase, T> func) => DBHelper.WithDb(func);
    public void WithDb(Action<LiteDatabase> action) => DBHelper.WithDb(action);
}
```

역할: DBHelper를 인터페이스로 추상화, 테스트 시 Mock 교체 가능

Layer 3: Repository (데이터 접근)

```
public class UserRepository : IUserRepository
{
    private readonly IDBGateway _db;
    private const string CUsers = "users";

    public UserRepository(IDBGateway db)
    {
        _db = db ?? throw new ArgumentNullException(nameof(db));
    }

    public bool ExistsEmail(string email) => _db.WithDb(db =>
    {
        var col = db.GetCollection<User>(CUsers);
        return col.Exists(u => u.Email == email);
    });

    public User FindActiveUserByEmail(string email) => _db.WithDb(db =>
    {
        var col = db.GetCollection<User>(CUsers);
        return col.FindOne(u => u.Email == email && u.IsActive);
    });
    // ...
}
```

Layer 4: Service (비즈니스 로직)

```
public class AuthService : IAuthService
{
    private readonly IUserRepository _users;
    private const int BcryptWorkFactor = 10;

    public AuthService(IUserRepository users)
    {
        _users = users ?? throw new ArgumentNullException(nameof(users));
        EnsureSuperAdmin(); // 기본 관리자 자동 생성
    }

    public Result<User> Login(string email, string password)
    {
        try
        {
            var e = AuthValidator.NormalizeEmail(email);
            if (!AuthValidator.IsValidEmail(e))
                return Result<User>.Fail(AuthError.EmailInvalid);

            var u = _users.FindActiveUserByEmail(e);
            if (u == null)
                return Result<User>.Fail(AuthError.NotFoundOrInactive);

            bool ok = BCrypt.Net.BCrypt.Verify(password, u.PasswordHash);
            if (!ok)
                return Result<User>.Fail(AuthError.PasswordMismatch);

            return Result<User>.Success(u);
        }
        catch (Exception ex)
        {
            Debug.LogError($"[AuthService] Login error: {ex}");
            return Result<User>.Fail(AuthError.Internal);
        }
    }
}
```

4.5 실제 데이터 흐름 예시

예시 1: 로그인 흐름

```
[LoginController]
| DataService.Instance.Auth.Login(email, password)
▼
[AuthService.Login()]
| _users.FindActiveUserByEmail(email)
▼
[UserRepository.FindActiveUserByEmail()]
| _db.WithDb(db => col.FindOne(u => u.Email == email && u.IsActive))
▼
[DBGateway.WithDb()] → [DBHelper.WithDb()] → [LiteDB]
| User 객체 반환 (역순)
▼
[LoginController]
| SessionManager.Instance.SignIn(user)
| 역할 기반 라우팅: USER → HOME, ADMIN+ → RESULT
▼
[SceneNavigator.GoTo() + Fade 효과]
```

예시 2: 보상 저장 흐름 (다중 Repository)

```
[CommonRewardStep.SaveReward()]
| ProblemContext.SaveReward(body, itemId, itemName)
▼
[LocalRewardService.SaveRewardForCurrentUser()]
|
├─ 1) Attempt 로그 저장
|   └─ LocalProgressService → ProgressRepository → LiteDB(attempts)
|
└─ 2) 인벤토리 아이템 지급
    └─ GrantInventoryItem(email, item)
        ├── UserRepository.FindActiveUserByEmail()
        └─ InventoryRepository.Add(item) → LiteDB(inventory)
```

4.6 인터페이스 분리 현황



5. 핵심 모듈별 상세 분석

5.1 부트스트랩 및 코어 시스템

Bootstrap.cs (앱 진입점)

```
Bootstrap.Start()
├─ DatabaseInitializer.InitializeIndexes() ← DB 인덱스 일괄 초기화
├─ LocalizedTable.Load(csvResource)      ← CSV 로컬라이제이션 로드
├─
└─ SceneManager 초기화 후:
    ├─ SessionManager.TryRestore() 성공 → HomeScene
    └─ 세션 없음 → RegisterScene
```

SessionManager.cs (세션 관리)

```
public class SessionManager : MonoBehaviour
{
    public static SessionManager Instance { get; private set; }
    public bool IsSignedIn { get; private set; }
    public User CurrentUser { get; private set; }
    public string SessionId { get; private set; }
    public event Action OnChanged;

    public void SignIn(User user);    // session.user, session.id를 PlayerPrefs에 저장
    public void SignOut();           // PlayerPrefs 삭제
    public bool TryRestore();         // 앱 재시작 시 세션 복원
}
```

SceneNavigator.cs (씬 전환)

```
public class SceneNavigator : MonoBehaviour
{
    public enum ScreenId { REGISTER, HOME, PROBLEM, RESULT }

    public void GoTo(ScreenId id);    // 인증 검사 + 비동기 로드 + 페이드 효과
    public void GoBack();             // 히스토리 스택에서 이전 씬으로
}
```

- 인증되지 않은 상태에서 HOME/PROBLEM/RESULT 접근 차단
- CanvasGroup 기반 페이드 인/아웃 전환 효과
- 씬 히스토리 스택으로 GoBack() 지원

GameManager.cs (앱 유틸리티)

```
public class GameManager : MonoBehaviour
{
    public static GameManager Instance { get; private set; }

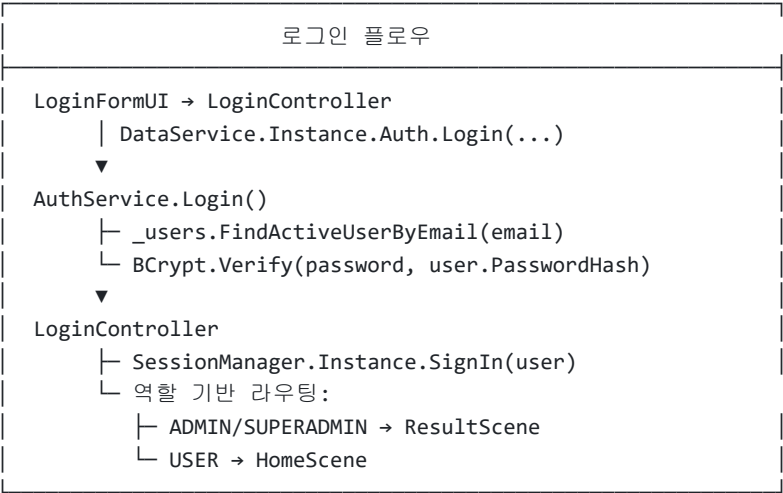
    public void QuitApplication();    // 앱 종료
    public void GoToHome();           // 홈으로 이동
    public void GoToScene(ScreenId screen); // 특정 씬으로
    public void Logout();             // 로그아웃 + 로그인 화면
}
```

5.2 인증 시스템 (RegisterScene + Service)

구성 요소

클래스	역할	패턴
LoginFormUI	로그인 입력 필드, 버튼 이벤트 발행	View
LoginController	AuthService 호출, 세션 설정, 씬 전환	Controller
SignupFormUI	회원가입 UI, 실시간 유효성 힌트	View
SignupController	이메일 중복 체크, 비밀번호 강도 표시	Controller
AuthService	BCrypt 해싱, 로그인/가입 로직	Service
AuthValidator	이메일 정규식, 비밀번호 검증 (6자+영문+숫자)	Utility
AuthConfig	기본 관리자 설정 (ScriptableObject)	Config
AuthUIText	한국어 UI 메시지 (ScriptableObject)	Config

인증 플로우



기본 관리자 자동 생성

```
// AuthService 생성자에서 호출
private void EnsureSuperAdmin()
{
    if (_users.HasSuperAdmin()) return;

    // AuthConfig (ScriptableObject)에서 설정 로드
    // 기본값: admin@local / admin1234
    var config = Resources.Load<AuthConfig>("AuthConfig");
    // SuperAdmin 계정 자동 생성
}
```

5.3 문제 풀이 시스템 (ProblemScene)

Step Base 클래스 계층

```
ProblemStepBase (abstract)
|
| 공통 기능:
| - OnStepEnter() / OnStepExit()
| - SaveAttempt(body) → ProblemContext 위임
| - SaveReward(body, itemId, itemName) → ProblemContext 위임
| - StepKey 생성 (StepKeyConfig 연동)
|
└─> MultipleChoiceStepBase<TQuestion>
    | 객관식 문제 처리
    | Abstract: QuestionCount, GetQuestion(), ApplyQuestionUI(),
    |           GetCorrectOptionIndex(), OnQuestionAttempted(),
    |           OnAllQuestionsCompleted()
    └─> Director_Problem3_Step1, Step3, ...

└─> RandomCardSequenceStepBase
    | 랜덤 카드 순서 처리
    | Abstract: CardCount, OnApplyCardToUI(), OnClearCurrentCardUI(),
    |           OnCardProcessed(), OnAllCardsProcessed()
    └─> Director_Problem1_Step2, ...

└─> InventoryDropTargetStepBase
    | 인벤토리 아이템 드래그 앤 드롭
    | DB에서 아이템 보유 확인 → 자동 활성화
    └─> Director_Problem4_Step2, Problem5_Step2, ...

└─> CommonRewardStep
    | 공통 보상 연출 (순차 애니메이션)
    | SaveReward() → DB 저장
    | MarkProblemSolved() → 완료 처리
    └─> 모든 문제의 Reward Step
```

ProblemContext (ScriptableObject)

```
[CreateAssetMenu]
public class ProblemContext : ScriptableObject
{
    public ProblemTheme Theme;
    public int ProblemIndex;
    public string ProblemId;
    public StepId CurrentStepKey;

    public void SaveStepAttempt(string body);
    public void SaveReward(string body, string itemId, string itemName);
}
```

문제 풀이 전체 흐름

```
HomeScene에서 문제 선택
|
| ProblemSession.CurrentTheme = Director
| ProblemSession.CurrentProblemIndex = 1
| SceneNavigator.GoTo(PROBLEM)
|
▼
ProblemSceneController.Start()
|
| SetupThemeRoot() // Director/Gardener 활성화
| ActivateSingleProblem() // 해당 Problem만 활성화
|
|
▼
StepFlowController.OnEnable()
| GoToStep(0) // Intro부터 시작
|
▼
```

```
Step 0: Intro Panel
| NextStep() 또는 SkipFlow()
|
▼
Step 1~3: 문제/인터랙션
| OnStepEnter()
| 사용자 인터랙션 (객관식/카드/드래그 등)
| SaveAttempt(body) → DB attempts 저장
| StepCompletionGate.MarkOneDone()
| NextStep()
|
▼
Reward Step: 보상 연출
| CommonRewardStep: 순차 애니메이션
| SaveReward() → DB attempts + inventory 저장
| ProblemEnd()
|
▼
StepFlowController.ProblemEnd()
| ProgressService.MarkProblemSolvedForCurrentUser()
|   ↳ DB results INSERT
| SceneNavigator.GoTo(HOME)
|   ↳ ProblemSession.ReturnTarget 기반 화면 결정
```

StepCompletionGate (진행도 위젯)

```
public class StepCompletionGate : MonoBehaviour
{
    public void ResetGate(int total);    // 필요 완료 수 설정
    public void MarkOneDone();           // 진행도 +1
    public void MarkOneUndone();         // 진행도 -1
    // UI: 프로그레스 바 + 완료 표시 + 자동 다음 Step 옵션
}
```

5.4 홈 씬 시스템 (HomeScene)

ProblemSession (정적 컨텍스트)

```
public static class ProblemSession
{
    public static ProblemTheme CurrentTheme;
    public static int CurrentProblemIndex;
    public static string CurrentProblemId;
    public static bool DemoMode = true;    // ← 기본 활성화: 모든 콘텐츠 잠금 해제

    public enum ReturnTarget { None, LevelSelect, Ending }
    public static ReturnTarget ReturnAfterProblem;
}
```

잠금 해제 로직

```
void RefreshSinglePanel(ThemePanelUI panel, ProblemTheme theme)
{
    var solvedSet = new HashSet<int>(
        DataService.Instance.Progress
            .FetchSolvedProblemIndexes(userEmail, theme).Value
    );

    bool[] unlocked = new bool[11];

    if (ProblemSession.DemoMode)
    {
        // 데모 모드: 전체 해제
        for (int i = 1; i <= 10; i++) unlocked[i] = true;
    }
    else if (solvedSet.Count >= 10)
    {
        // 전체 완료: 자유 선택
        for (int i = 1; i <= 10; i++) unlocked[i] = true;
    }
    else
    {
        // 순차 진행: 다음 미해결 문제만 해제
        int nextIndex = FindFirstNotSolved(solvedSet);
        for (int i = 1; i <= 10; i++) unlocked[i] = (i == nextIndex);
    }

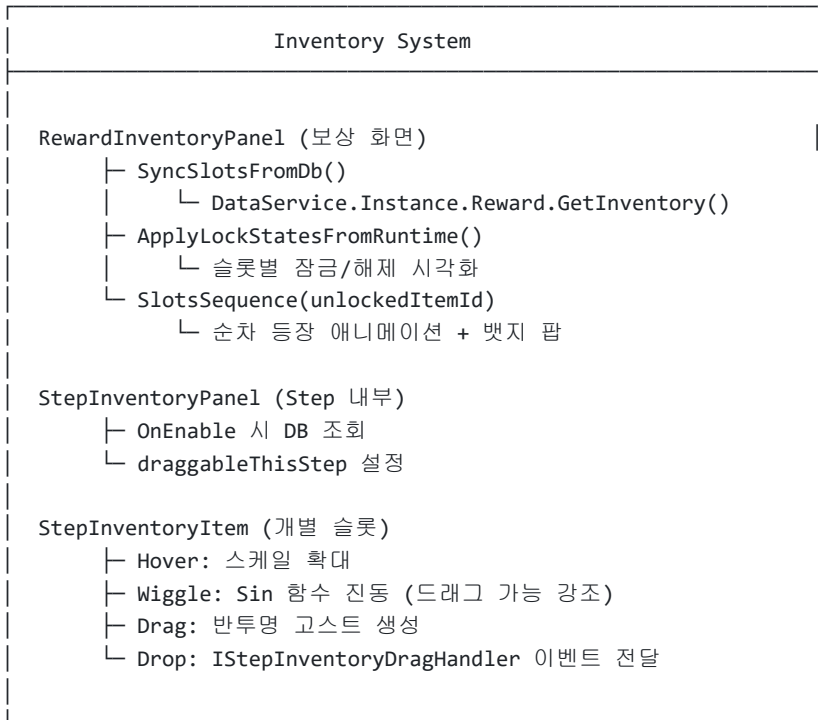
    panel.SetLockStates(unlocked, solvedSet);
}
```

ReturnTarget 기반 홈 화면 분기

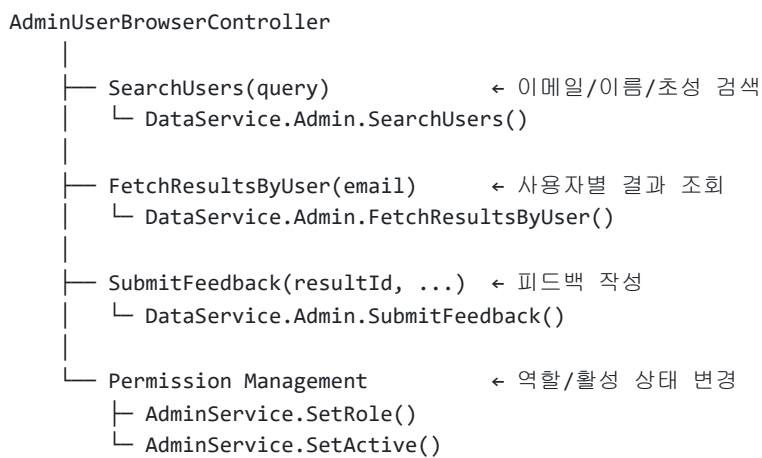
ProblemScene에서 복귀 시:

- |
- ├ ReturnTarget.None → 테마 선택 화면
- ├ ReturnTarget.LevelSelect → Director 레벨 선택 (P1-9 완료 시)
- └ ReturnTarget.Ending → 엔딩 패널 (P10 완료 시)

5.5 인벤토리 시스템



5.6 관리자 시스템 (ResultScene)



6. 데이터 모델 및 스키마

6.1 Collection 스키마

users Collection

필드	타입	설명	인덱스
Id	string (GUID)	고유 식별자	Unique
Email	string	로그인 ID	Unique
PasswordHash	string	BCrypt 해시	-
Name	string	사용자 이름	-
LowerName	string	소문자 이름 (검색용)	Index
NameChosung	string	초성 (ㄱㅈㅊ)	Index
Role	UserRole	USER/ADMIN/SUPERADMIN	Index
IsActive	bool	활성 상태	-
CreatedAt	DateTime	생성일	-

results Collection

필드	타입	설명	인덱스
Id	string (GUID)	고유 식별자	-
UserId	string	users.Id FK	Index
Theme	string	"Director"/"Gardener"	Index
ProblemIndex	int	1~10	Index
Score	int	점수	-
CorrectRate	decimal?	정답률 (0~1)	-
DurationSec	int?	풀이 시간(초)	-
MetaJson	string	추가 메타데이터	-
CreatedAt	DateTime	생성일	-

attempts Collection

필드	타입	설명	인덱스
Id	string (GUID)	고유 식별자	Unique
UserId	string	사용자 ID	Index
UserEmail	string	사용자 이메일	Index
SessionId	string	세션 ID	-
ProblemId	string	문제 ID	Index
Theme	ProblemTheme	Director/Gardener	Index
ProblemIndex	int?	1~10	Index
Content	string	JSON payload	-
CreatedAt	DateTime	생성일	-

inventory Collection

필드	타입	설명	인덱스
Id	string (GUID)	고유 식별자	-
UserId	string	사용자 ID	-
UserEmail	string	사용자 이메일	Index
ItemId	string	"mind_lens" 등	-
ItemName	string	"마음 렌즈" 등	-
Theme	ProblemTheme	테마	-
ProblemIndex	int	획득 문제 번호	-
AcquiredAt	DateTime	획득일	-

feedback Collection

필드	타입	설명	인덱스
Id	string (GUID)	고유 식별자	-
ResultId	string	results.Id FK	Index
AdminEmail	string	관리자 이메일	-
Comment	string	피드백 내용	-
Score	int	평가 점수	-
CreatedAt	DateTime	생성일	-

sessions Collection

필드	타입	설명	인덱스
Id	string (GUID)	고유 식별자	-
UserId	string	사용자 ID	-
UserEmail	string	사용자 이메일	-
Theme	ProblemTheme	테마	-
CurrentStep	string	현재 Step	-
CreatedAt	DateTime	생성일	-

problems Collection

필드	타입	설명	인덱스
Id	string (GUID)	고유 식별자	-
OwnerEmail	string	출제자 이메일	-
Theme	ProblemTheme	Director/Gardener	Index
Index	int	1~10	Index
Title	string	문제 제목	-
Content	string	문제 내용	-

6.2 데이터 모델 Enum

```
public enum UserRole { USER = 0, ADMIN = 1, SUPERADMIN = 2 }
public enum ProblemTheme { Director = 0, Gardener = 1 }
public enum StepId { Step1, Step2, Step3, Step4, Reward, Extra1, Extra2 }

public enum AuthError
{
    None, NameEmpty, EmailInvalid, EmailDuplicate, PasswordWeak,
    NotFoundOrInactive, PasswordMismatch, Internal, InventoryError
}
```

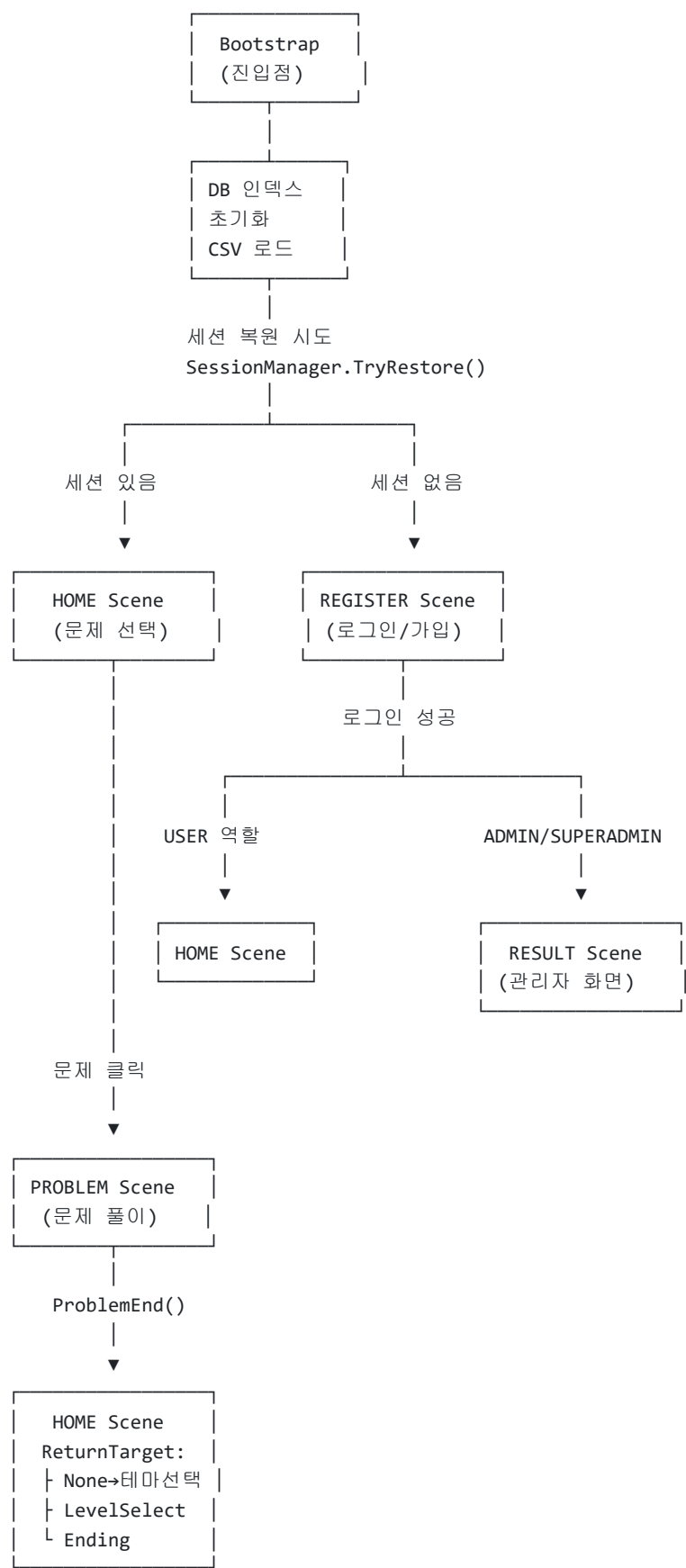

6.3 보조 데이터 모델

```
// 사용자 진행도 요약
public class UserProgress
{
    public string UserEmail;
    public int TotalSessions;
    public int TotalSolved;
    public DateTime? LastSessionAt;
}

// 사용자 검색 결과
public class UserSummary
{
    public string Email;
    public string Name;
    public UserRole Role;
    public bool IsActive;
}
```

7. 씬 및 UI 플로우

7.1 전체 씬 전환 다이어그램



7.2 싹별 ScreenId

```
public enum ScreenId
{
    REGISTER,    // RegisterScene - 로그인/회원가입
    HOME,        // HomeScene - 테마 선택, 문제 선택
    PROBLEM,     // ProblemScene - 문제 풀이
    RESULT       // ResultScene - 관리자 대시보드
}
```

8. 부가 시스템

8.1 이펙트 시스템 (66개 파일)

공통 이펙트 (Effect/Common/)

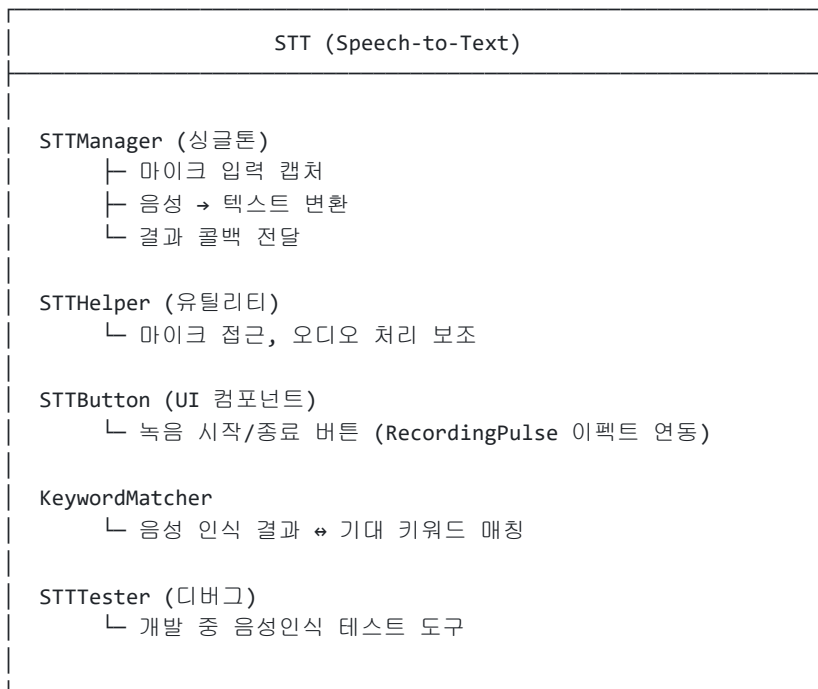
이펙트	설명
EffectControllerBase	이펙트 컨트롤러 추상 베이스
AppearAnimation	요소 등장 애니메이션
BadgeGlowPulse	배지 발광 맥동
ButtonHover	버튼 호버 스케일 효과
CardFlip	카드 뒤집기 3D 효과
DragScaleFeedback	드래그 시 스케일 피드백
DropZoneIndicator	드롭 영역 시각 표시
DustParticleSpawner	먼지 파티클 생성
EmotionGlowSync	감정 표현 발광 동기화
FloatingAnimation	떠다니는 효과 (Sin 함수)
GlowEffect	범용 발광 효과
IntroElement	인트로 요소 연출
PopupImageDisplay	팝업 이미지 표시
PopupSpring	스프링 물리 팝업
PulseAnimation	맥동 애니메이션
RadialBurstEffect	방사형 폭발 파티클
RecordingPulse	녹음 중 맥동
RewardParticleBurst	보상 파티클 폭발
SelectionGlow	선택 항목 발광
ShakeTrigger	흔들림 트리거
SlideUpFadeIn	아래→위 슬라이드 페이드인
SparkleEffect / SparkleParticle	반짝임 효과
SpriteSwapButton	스프라이트 교체 버튼
SweepHighlightTrigger	스융프 하이라이트
TextOpacityPulse	텍스트 투명도 맥동
TouchPromptPulse	터치 유도 맥동

문제별 이펙트 (Effect/ProblemScene/Director/)

```
Director/  
├─ Problem1/  
│   ├── Problem1_Step1_EffectController.cs  
│   ├── Problem1_Step2_EffectController.cs  
│   └── Problem1_Step3_EffectController.cs  
├─ Problem2/ ~ Problem10/  
└─ ... (각 문제별 2~3개 Step 이펙트 컨트롤러)
```

각 문제의 Step별로 고유한 시각 효과를 제어하며, EffectControllerBase 를 상속하여 일관된 인터페이스를 유지합니다.

8.2 음성인식 시스템 (STT)

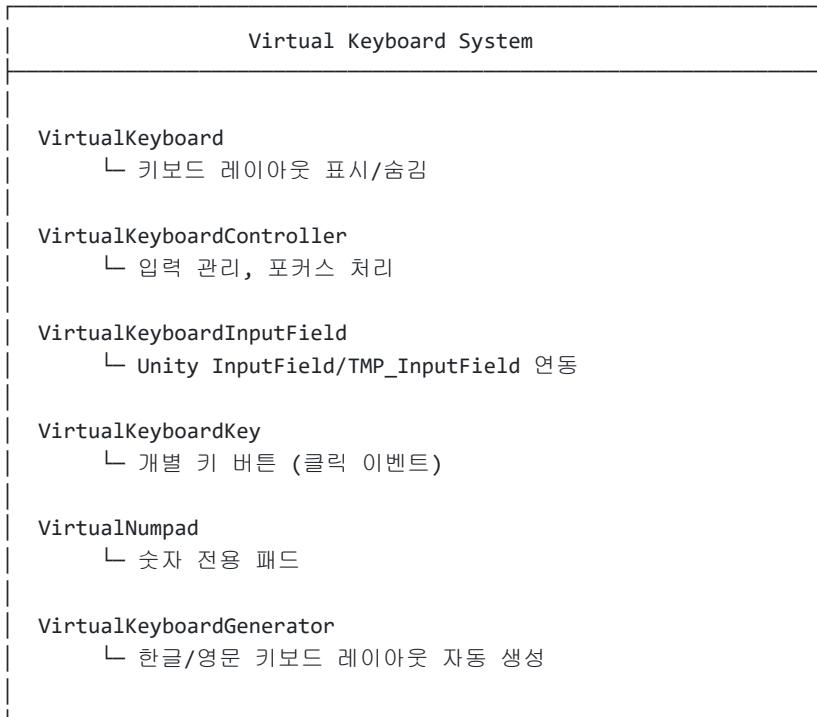


8.3 사운드 시스템

```
public class SoundManager : MonoBehaviour  
{  
    public static SoundManager Instance { get; private set; }  
  
    public void PlayBGM(string clipName);    // Resources/BGM/{clipName}  
    public void StopBGM();  
}
```

- StepFlowController에서 Step 전환 시 BGM 제어
- 문제별 고유 BGM 지원

8.4 가상 키보드 시스템



8.5 로컬라이제이션 시스템

```
public static class LocalizedTable
{
    // Bootstrap에서 CSV 로드
    public static void Load(string csvResource);

    // 텍스트 ID로 번역 조회
    public static string Get(string textId, bool korean = true);
}
```

- **CSV 파일:** Resources/CSV/MC_DataTable_v01
- **지원 언어:** 한국어 / 영어
- Bootstrap 시작 시 일괄 로드

8.6 한글 초성 검색 (UserSearchUtility)

```
public static class UserSearchUtility
{
    // "ㄱㅈㅍ"으로 "김철수" 검색
    public static string ExtractChosung(string name);
    public static bool MatchesChosung(string nameChosung, string query);
}
```

관리자 화면에서 사용자 이름을 초성으로 빠르게 검색할 수 있도록 지원합니다.

8.7 디버그 도구

클래스	기능
DebugShortcutController	개발용 단축키 (씬 이동, 데이터 초기화 등)
DebugStepController	Step 네비게이션 (이전/다음 Step 즉시 이동)

9. 적용된 디자인 패턴

9.1 패턴 목록

패턴	적용 위치	설명
Singleton	DataService, SessionManager, SceneNavigator, GameManager, SoundManager	앱 전역 상태 관리
Repository	모든 Repository 클래스 (6개)	데이터 접근 추상화
Service Layer	AuthService, ProgressService, RewardService 등 (6개)	비즈니스 로직 캡슐화
MVC	RegisterScene (FormUI + Controller)	UI와 로직 분리
Template Method	ProblemStepBase, EffectControllerBase	공통 흐름 정의, 세부 구현 위임
Strategy	MultipleChoiceStepBase<T>	제네릭으로 알고리즘 교체
Observer	이벤트 기반 UI 통신, SessionManager.OnChanged	느슨한 결합
Facade	DataService	복잡한 하위 시스템 단순화
Result Monad	Result<T>	명시적 에러 처리
ScriptableObject Config	AuthConfig, AuthUIText, ProblemContext	에디터 설정 분리

9.2 주요 패턴 상세

Singleton 패턴 (DontDestroyOnLoad)

```
public class DataService : MonoBehaviour
{
    public static DataService Instance { get; private set; }

    void Awake()
    {
        if (Instance != null) { Destroy(gameObject); return; }
        Instance = this;
        DontDestroyOnLoad(gameObject);
        // ... DI 조립
    }
}
```

Result Monad 패턴

```
public readonly struct Result<T>
{
    public bool Ok { get; }
    public AuthError Error { get; }
    public string Message { get; }
    public T Value { get; }

    public static Result<T> Success(T value) => new(true, default, null, value);
    public static Result<T> Fail(AuthError error) => new(false, error, null, default);
}

// 사용 예시
var result = AuthService.Login(email, password);
if (!result.Ok) { ShowError(result.Error); return; }
var user = result.Value;
```

ScriptableObject 설정 패턴

```
[CreateAssetMenu]
public class AuthConfig : ScriptableObject
{
    public string DefaultAdminEmail = "admin@local";
    public string DefaultAdminPassword = "admin1234";
    public string DefaultAdminName = "관리자";
}
```

10. 보안 및 검증

10.1 비밀번호 보안

```
// BCrypt 해싱 (WorkFactor = 10, 2^10 = 1024 라운드)
string hash = BCrypt.Net.BCrypt.HashPassword(password, 10);

// 검증
bool isValid = BCrypt.Net.BCrypt.Verify(password, user.PasswordHash);
```

- 평문 비밀번호 저장 없음
- Salt 자동 생성 및 해시에 포함
- WorkFactor=10 (보안 강도와 성능 균형)

10.2 입력 검증

```
public static class AuthValidator
{
    private static readonly Regex EmailRx = new Regex(
        @"^[A-Za-z0-9._%+\-]+\@[A-Za-z0-9.\-]+\.[A-Za-z]{2,}$");

    public static bool IsValidEmail(string email);

    // 비밀번호 강도: 6자 이상 + 영문 + 숫자
    public static bool IsStrongPassword(string pw);

    // 이메일 정규화: trim + lowercase
    public static string NormalizeEmail(string email);
}
```

10.3 권한 관리

```
// 역할 기반 라우팅
if (user.Role >= UserRole.ADMIN)
    SceneNavigator.GoTo(ScreenId.RESULT); // 관리자 화면
else
    SceneNavigator.GoTo(ScreenId.HOME);    // 일반 사용자 화면

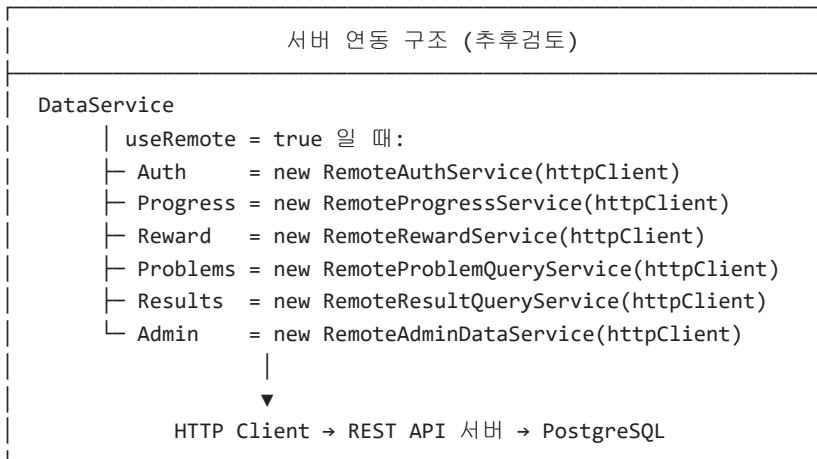
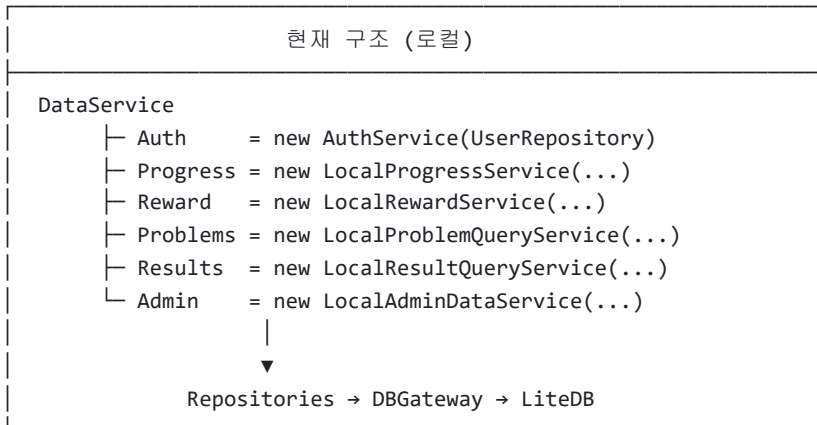
// 관리자 기능 보호
public static class AdminService
{
    public static void SetRole(string targetEmail, UserRole newRole);
    public static void SetActive(string targetEmail, bool active);
    // 호출자가 ADMIN+ 권한인지 검증
}
```

10.4 세션 관리

- **저장소:** PlayerPrefs (로컬 개발용)
- **세션 ID:** GUID 기반
- **세션 복원:** 앱 재시작 시 PlayerPrefs에서 복원 시도
- **제한사항:** PlayerPrefs는 평문 저장이므로 프로덕션 환경에서는 보안 강화 필요

11. 서버 연동 가이드

11.1 현재 로컬 구조 vs 서버 연동 구조



11.2 수정이 필요한 파일들

1단계: 원격 Service 구현 생성

```
Assets/01. Script/Data/
├─ Remote/                                # 새로 생성
│   ├── RemoteAuthService.cs
│   ├── RemoteProgressService.cs
│   ├── RemoteRewardService.cs
│   ├── RemoteProblemQueryService.cs
│   ├── RemoteResultQueryService.cs
│   ├── RemoteAdminDataService.cs
│   ├── HttpClientWrapper.cs           # HTTP 통신 래퍼
│   └─ TokenManager.cs                 # JWT 토큰 관리
```

2단계: DataService 수정

```
public class DataService : MonoBehaviour
{
    [SerializeField] bool useRemote = false;
    [SerializeField] string baseUrl = "https://api.example.com";

    void Awake()
    {
        // 싱글톤 설정...

        if (useRemote)
            InitializeRemoteServices();
        else
            InitializeLocalServices();
    }

    private void InitializeLocalServices()
    {
        // 현재 코드 그대로
    }

    private void InitializeRemoteServices()
    {
        var httpClient = new HttpClientWrapper(baseUrl);
        Auth      = new RemoteAuthService(httpClient);
        Progress  = new RemoteProgressService(httpClient);
        Reward    = new RemoteRewardService(httpClient);
        // ...
    }
}
```

11.3 서버 API 설계 예시

Method	Endpoint	설명	Request Body	Response
GET	/api/auth/exists?email={email}	이메일 중복 확인	-	{ exists: bool }
POST	/api/auth/signup	회원가입	{ name, email, password }	{ success: bool }
POST	/api/auth/login	로그인	{ email, password }	{ token, user }
GET	/api/progress/{userEmail}	진행도 조회	-	{ progress }
GET	/api/progress/{userEmail}/solved	해결한 문제	-	{ indexes: int[] }
POST	/api/progress/attempt	Attempt 저장	{ attempt }	{ success: bool }
POST	/api/progress/solve	문제 완료	{ theme, problemIndex }	{ success: bool }
GET	/api/inventory/{userEmail}	인벤토리 조회	-	{ items: [] }
POST	/api/inventory/grant	아이템 지급	{ item }	{ success: bool }

11.4 수정 체크리스트

- [] DataService.cs - useRemote 분기 처리 추가
- [] Remote/HttpClientWrapper.cs - HTTP 통신 래퍼 생성
- [] Remote/TokenManager.cs - JWT 토큰 관리
- [] Remote/RemoteAuthService.cs - 원격 인증 서비스
- [] Remote/RemoteProgressService.cs - 원격 진행도 서비스
- [] Remote/RemoteRewardService.cs - 원격 보상 서비스
- [] Remote/RemoteProblemQueryService.cs - 원격 문제 조회
- [] Remote/RemoteResultQueryService.cs - 원격 결과 조회
- [] Remote/RemoteAdminDataService.cs - 원격 관리자 서비스
- [] SessionManager.cs - 토큰 기반 세션 관리 추가
- [] 서버 측 REST API 구현
- [] 서버 측 데이터베이스 스키마 설계

12. 결론 및 개선 제안

12.1 프로젝트 강점

1. **명확한 계층 분리:** UI → Controller → Service → Repository → DB
2. **의존성 역전 원칙 준수:** 인터페이스 기반 설계로 로컬 ↔ 서버 교체 용이
3. **확장 가능한 설계:** 새로운 문제 타입, 테마 추가 용이 (Step Base 클래스)
4. **풍부한 이펙트 시스템:** 66개 이펙트로 몰입감 있는 사용자 경험

5. 음성인식 통합: STT 모듈로 음성 기반 인터랙션 지원
6. 일관된 네이밍: 클래스/메서드명이 역할을 명확히 표현
7. 재사용 가능한 컴포넌트: StepCompletionGate, EffectControllerBase, Step Base 클래스들
8. ScriptableObject 활용: 설정 분리로 에디터에서 쉽게 조정 가능
9. DB 인덱스 중앙 관리: DatabaseInitializer로 성능 최적화

12.2 개선 제안

현재	제안	이유
동기 DB 호출	비동기 처리 도입	메인 스레드 블로킹 방지
정적 ProblemSession	ScriptableObject 또는 DI	테스트 용이성
Exception + Result 혼용	Result 패턴 통일	일관된 에러 처리
스키마 변경 시 Drop	마이그레이션 전략	데이터 보존
PlayerPrefs 세션 저장	암호화된 세션 저장	보안 강화
DemoMode 기본 활성화	프로덕션 빌드 시 비활성화	릴리스 안전성

12.3 기술적 성과 요약

- 총 210개 C# 스크립트를 체계적으로 구조화 (v1 대비 92개 → 210개로 확장)
- LiteDB + BCrypt를 활용한 안전한 로컬 데이터 관리
- 인터페이스 기반 설계로 로컬 ↔ 서버 전환 용이
- 제네릭 Base 클래스로 코드 중복 최소화
- 이벤트 기반 아키텍처로 낮은 결합도 유지
- 66개 이펙트 시스템으로 풍부한 시각적 피드백
- STT 음성인식 통합으로 다양한 인터랙션 지원
- 가상 키보드 시스템으로 터치 기반 입력 지원
- CSV 기반 다국어 로컬라이제이션 지원
- DB 인덱스 중앙 관리 (DatabaseInitializer)로 성능 최적화
- ScriptableObject 기반 설정 관리로 유연한 구성

12.4 중간보고 대비 주요 변경 사항

항목	v1 (2025-11-28)	v2 (2026-03-11)
총 스크립트 수	92개	210개
코어 시스템	SessionManager, SceneNavigator	+ GameManager, Core 폴더 분리
DB 초기화	Repository별 EnsureIndex	DatabaseInitializer 중앙 관리
인증 설정	하드코딩	AuthConfig ScriptableObject
비밀번호 최소 길이	8자	6자
이펙트	7개	66개 (공통 25+ / 문제별 40+)
음성인식	없음	STT 모듈 (5개 파일)
가상 키보드	없음	Keyboard 모듈 (6개 파일)
사운드	없음	SoundManager (BGM 관리)
로컬라이제이션	없음	CSV 기반 한국어/영어 지원
디버그 도구	없음	DebugShortcut, DebugStep
서비스 인터페이스	4개	6개 (+IProblemQueryService, IResultQueryService)
ProblemSession	기본	+ DemoMode, ReturnTarget
ProblemContext	없음	ScriptableObject 기반

문서 작성: 서강대 가상융합전문대학원 산학협력단 **분석 대상:** Hanam_MC Unity 프로젝트 (하남시정신건강복지센터 AI생각돌
봄 소프트웨어) **총 스크립트 수:** 210개 **주요 기술:** Unity, C#, LiteDB, [BCrypt.Net](#), TextMeshPro, STT/TTS 등